

Using Codex Effectively in Cursor and Structuring a Project Root Intelligently

Table of Contents

Using Codex Effectively in Cursor and Structuring a Project Root Intelligently

Executive Summary

The most effective approach in 2026 is not to treat Cursor and the Codex extension as competitors, but as a complementary stack inside the same editor. The Codex extension works in Cursor because it targets VS Code-compatible editors, uses the same agent as Codex CLI, and shares the same configuration as that CLI. Cursor, in turn, provides its own mechanisms that are very useful for agentic development: project rules in `.cursor/rules/*.mdc`, skills, Plan Mode, checkpoints, message queues, worktrees, cloud agents, and code-search tools. This combination is especially strong when you clearly separate durable guidance, Cursor-specific rules, reusable workflows, and execution configuration.

The golden rule is simple: put repository invariants in `AGENTS.md`, keep Cursor rules small and focused in `.cursor/rules/`, package repeated workflows as `SKILL.md`, and centralize Codex behavior in `.codex/config.toml`. OpenAI explicitly recommends `AGENTS.md` as durable project guidance, advises keeping it short and practical, and suggests turning recurring prompts into skills. Cursor recommends the same pattern for agents: essential rules, references to canonical files rather than duplication, and incremental additions only when recurrent errors appear.

For daily work, the best results come from short, verifiable workflows: open the right files, provide minimal but precise context, ask for a plan before a large change, run the relevant tests, and then review the diff. Codex recommends structuring prompts in four parts - goal, context, constraints, and definition of done - and the IDE extension already injects open files and the active selection, which allows shorter prompts. Cursor also recommends avoiding excessive file tagging, letting the agent find context when possible, and starting a new conversation when the thread becomes noisy or the task changes.

Security and reliability should remain strict by default. On the Codex side, the standard mode limits the agent to the workspace, disables network access by default, and requires approvals when the agent needs to leave the sandbox or access the network. On the Cursor side, sensitive actions remain protected by guardrails, Auto-review becomes the recommended mode in recent versions, and `.cursorignore` can block agent access to selected files. In both cases, web results should be treated as untrusted by default; review diffs locally; and rely on Git, tests, linting, and CI rather than only natural-language instructions.

A final important point: recent Codex updates reinforce exactly this style of work. OpenAI announced in May 2026 a generalized Goal Mode in the Codex app, CLI, and IDE extension, as well as richer context for long-running tasks. In practice, this encourages a more teammate-like workflow: you provide a durable objective, success criteria, checks, plans, and stable reference points in the repository.

Starting Assumptions and Recommended Architecture

You asked me to assume there are no specific constraints regarding project type, programming language, or team size. I therefore assume a general-purpose software repository, possibly a mono-repo, used by both a human team and AI agents, with a desire to remain portable between Cursor, Codex, and, if needed, other agents compatible with AGENTS.md. This assumption favors a repository architecture that is tool-agnostic at the center and tool-specific at the edges.

The recommended mental architecture is the following:

- AGENTS.md contains the durable rules shared across the repository.
- .cursor/rules/*.mdc contains Cursor-specific rules.
- .agents/skills/, .cursor/skills/, or .codex/skills/ contains reusable workflows.
- .codex/config.toml contains project-level Codex preferences.
- Human documentation such as README.md, ARCHITECTURE.md, CONTRIBUTING.md, CODING-STANDARDS.md, WORKFLOWS.md, SECURITY.md, and CHANGELOG.md serves as reference material that the agent can read and cite.
- Repository hygiene files such as .editorconfig, .cursorignore, .cursorindexingignore, CODEOWNERS, and LICENSE maintain consistency and governance.

For a Cursor + Codex stack, I recommend using AGENTS.md as the shared canonical source and .agents/skills/ as a neutral location when portability matters. Cursor explicitly detects .agents/skills/, .cursor/skills/, and .codex/skills/, including for compatibility with Claude and Codex. OpenAI also shows repo-local usage of .agents/skills/ in Agents SDK repositories. If your project is very Cursor-first, .cursor/skills/ is perfectly valid. If it is very Codex-first, .codex/skills/ is also valid. For a repository shared across multiple tools, .agents/skills/ is the best compromise.

flowchart TD

```
A[AGENTS.md] --> B[Human docs]
A --> C[Reusable skills]
A --> D[Plans and workflows]
E[.cursor/rules/*.mdc] --> F[Cursor Agent]
C --> F
C --> G[Codex extension]
H[.codex/config.toml] --> G
I[.cursorignore / .cursorindexingignore] --> F
B --> F
```

B --> G
J[CI / PR / review] --> B
J --> C
J --> A

This graph reflects a design recommendation: a shared base readable by humans and AI, plus tool-specific layers where necessary. This is the most robust way to avoid duplicating instructions between Cursor and Codex while keeping the advantages of both.

When to Use Native Cursor, Codex, or Both

Situation	Best entry point	Why
Quickly understand an unfamiliar repository	Cursor Agent or Codex IDE	Both can read, search, and summarize the codebase. Codex benefits from editor context, while Cursor has strong semantic search and proven exploration patterns.
Multi-file change with a prior plan	Cursor Plan Mode, then Codex for implementation	Cursor Plan Mode structures the research phase well. Codex is excellent at following a goal, constraints, and a local/cloud diff.
Long refactor, possibly in parallel	Codex cloud + Goal Mode, or Cursor worktrees	Codex keeps context between local and cloud work; Cursor and Codex both provide mechanisms for parallel work.
Recurring release, review, docs, or verification tasks	Skills + CI	OpenAI recommends turning these workflows into skills and later moving them into CI using GitHub Action.
Visual/UI debugging	Cursor for the local loop, Codex for image context and longer goals	Cursor is strong for browser and screenshot iteration. Codex accepts images, annotations, and prototype prompts.

Practical Workflows, Intelligent Prompts, and Pair-Programming Patterns

The best prompting is not the longest prompting; it is the prompting that constrains the work correctly. OpenAI recommends the default structure: Goal / Context / Constraints / Done when. In Cursor, the agent can often find context itself through semantic search and grep. In Codex, the IDE extension already injects open files and the active selection, so you can remain concise. The practical consequence is important: describe the expected result precisely instead of pasting the entire repository.

Prompt Formula to Use Almost Every Time

Goal

- What I want to achieve.

Context

- Relevant files, impacted flow, error, screenshot, PR, or issue.
- References: @fileA @fileB

Constraints

- Keep public API stable / no new dependency / no schema change / keep existing style.

Definition of done

- Tests X and Y pass
- Lint/typecheck pass
- Diff reviewed
- Manual verification described

This structure increases reliability because it turns a vague request into a verifiable contract.

Concrete Workflows to Prefer

Workflow	Recommended prompt or action	Why it works
Understand a flow	Explain the request flow through the selected code. Summarize the responsibilities of each module, validations, and 2 pitfalls to watch for.	Useful for onboarding and reverse engineering. It asks the agent the same questions you would ask a teammate.
Reproducible bug	Provide the repro recipe + constraints + request a rerun after the fix	Exact repro steps make the fix more reliable. Cursor Debug Mode is useful when the bug requires instrumentation and hypotheses.
TDD	Write the tests first. Confirm they fail. Do not code the implementation at this stage. Then: Implement without modifying the tests; loop until green.	The loop keeps the task grounded in observable behavior.
UI prototype from a screenshot	Add the image and provide framework + visual constraints + deliverables	Both Codex and Cursor support image context. This is useful for design-to-code

Workflow	Recommended prompt or action	Why it works
Careful refactor	/plan or Plan Mode, then: Implement Milestone 1 only. Keep the public API stable. Add a rollback strategy.	work. Large changes become safer when split into milestones.
Local review before commit	/review, then /review Focus on edge cases and security issues	Codex exposes review mode, and the same pattern can standardize code review.

Useful Prompt Templates

Planning Template

I want an implementation plan before any modification.

Goal

- [feature or refactor]

Context

- First read @fileA @fileB and the folder @src/moduleX
- Refer to AGENTS.md and WORKFLOWS.md

Constraints

- No public API change
- No new dependency
- Preserve existing conventions

Deliverable

- Step-by-step plan with impacted files
- Risks
- Rollback strategy
- Tests/checks to run

Wait for my approval before writing code.

This template uses what Cursor and Codex do best when asked to be careful: preliminary search, clarification, and a reviewable plan.

Bug-Fix Template

Bug

- [observable symptom]

Reproduction

1. ...
2. ...
3. ...

Context

- Suspects: @fileA @fileB
- Also inspect calls from @caller.ts

Constraints

- Minimal fix
- Do not change the API shape
- Add a regression test if feasible

Done when

- You reproduce the bug locally
- You propose a patch
- You rerun the repro
- You run the smallest relevant set of checks
- You report the commands and results

Pair-Programming Template

Act as my senior pair programmer.

Current step

- [what I am trying to do]

What I expect from you

- Challenge my assumptions
- Ask at most 3 clarification questions
- Propose 2 approaches
- Recommend 1 with trade-offs
- Do not edit code yet until we choose the approach

Asking Codex to interview you when the idea is still unclear produces better plans and avoids premature implementation.

Effective Pair-Programming Patterns

The most robust pattern is **Architect -> Implementer -> Reviewer**. In practice, this means: first create a plan, then execute one milestone, then run a local or PR review. OpenAI shows examples of specialized subagents such as `pr_explorer`, `reviewer`, and `docs_researcher`; Cursor also allows the same prompt to be run across multiple models side by side to compare approaches. For a team, this becomes a systematic second pair of eyes, even when working alone.

A second very effective pattern is **Explorer -> Fixer -> Verifier**. The explorer changes nothing; it maps the code. The fixer edits. The verifier reviews the diff, runs checks, and challenges assumptions. This pattern matches the logic of subagents, review modes, and TDD loops.

Security, Guardrails, Context Management, Memory, Multi-File Work, and CI

Guardrails and Permissions

For Codex, a good project baseline is to remain in Agent mode or a workspace-write sandbox with on-request approvals until the repository is fully understood. Network access is disabled by default. The agent can work in the active directory but must ask for approval if it leaves the workspace or needs network access. Keep the sandbox and permissions strict by default, and loosen them only for trusted repositories or specific workflows.

For Cursor, the logic is similar: sensitive actions require approval, recent versions support Auto-review, and `.cursorignore` can block agent access to selected files. Cursor also reminds users that file modifications are saved to disk and that Git should be the durable rollback mechanism. In short: never treat the agent as your only security barrier.

Managing the Context Window

The healthiest way to manage context is to separate four layers.

The first layer is **ephemeral working context**: open files, active selection, `@file`, image, issue, and PR. Codex IDE uses this directly; Cursor can also use open files, search, and the current branch.

The second layer is **durable repository context**: `AGENTS.md`, workflow docs, standards, and architecture. `AGENTS.md` should remain short and practical, while specialized documents should be referenced from it instead of copied into it entirely. Cursor rules should follow the same principle: short, focused, and linked to canonical files.

The third layer is **context loaded on demand**: skills, subagents, and specialized docs. This is the value of skills: the system initially loads only the name and description, and loads the complete `SKILL.md` only when needed.

The fourth layer is **historical context**: previous conversations, memories, and summaries. Start a new conversation when the task changes or the agent becomes unfocused. Use `/compact`, `/resume`, `/fork`, `/status`, or past-chat references instead of forcing a long noisy thread to continue. Memories are local and optional; they must not replace mandatory versioned instructions in the repository.

Memory and Cache Strategies

In Codex, **Memories** are a local recall layer for stable preferences, recurring conventions, known pitfalls, and frequent workflows. They are stored under `~/.codex/memories/` and can be enabled with:

[features]

memories = true

However, team guidance should still live in AGENTS.md or versioned docs. Memories are a local bonus, not a shared source of truth.

For performance and stability, several settings in .codex/config.toml are worth knowing:

- model_auto_compact_token_limit to trigger compaction.
- tool_output_token_limit to prevent command output from flooding the context.
- project_doc_max_bytes to limit how much of AGENTS.md is injected on the first turn.
- features.shell_snapshot, enabled by default, to speed up repeated shell commands.

In practice, this justifies keeping the main AGENTS.md very short and moving details into targeted reference docs.

For web use, the local Codex extension uses a web-search cache maintained by OpenAI by default. This reduces exposure to live prompt injection, but web results should still be treated as unsafe unless verified.

Multi-File Work, Parallel Work, and Refactors

Cursor and Codex can both handle multi-file changes, but you must avoid concurrency confusion. You can run several threads in parallel, but two threads should not modify the same files. Cursor provides native worktrees to isolate changes by agent, and Codex also supports worktrees for parallel execution without polluting local branches.

Recommended practice: **one agent per file surface**, or separate worktrees. For large refactors, plan locally, implement one milestone at a time, and delegate to the cloud only when the plan is stable.

Tests, Review, and CI Integration

The right pattern is: write or adjust tests, run the smallest relevant set of checks, review the diff, then launch a structured review. Do not stop at “make the change.” Also ask the agent to create tests, run checks, confirm results, and review.

For CI, OpenAI provides a Codex GitHub Action (openai/codex-action@v1) for CI/CD jobs, patches, and reviews posted by GitHub Actions. Codex can also perform PR reviews through GitHub. The mature approach is to keep workflow logic in the repository as skills first, then move it into CI when it becomes stable.

flowchart LR

```
A[Plan or /plan] --> B[Local implementation]
B --> C[Targeted tests]
C --> D[Local /review]
D --> E[GitHub PR]
E --> F[Codex review / CI]
```


F --> G[Merge]

G --> H[Update docs and project memory]

This loop formalizes the recommended behavior: plan, execute, test, review, move to PR/CI, then update docs continuously rather than later.

Useful Shortcuts and Commands in Cursor + Codex

Action	Shortcut / command	Recommended use
Open Cursor Agent	Cmd/Ctrl + I	Main entry point for exploration, planning, and editing in Cursor.
Activate Plan Mode in Cursor	Shift + Tab in the agent input	Use for every risky or multi-file change.
Queue a message while Cursor is working	Enter	Useful for preparing the next step without interrupting the current run.
Send immediately in Cursor	Cmd/Ctrl + Enter	Use only when you want to interrupt the queue.
Open command palette	Cmd/Ctrl + Shift + P	Required for advanced Codex and Cursor commands.
New Codex thread	Cmd/Ctrl + N	Good habit when the context is too long or the task changes.
Add selection to Codex thread	chatgpt.addToThread	Useful for asking for a test, explanation, or targeted refactor.
Add a complete file to Codex thread	chatgpt.addFileToThread	Use when the entire file is genuinely relevant.
Check context/limits in Codex	/status	Use when a thread is growing or quotas may be close.
Switch local/cloud in Codex	/local / /cloud	Move from local exploration to longer cloud execution.
Switch to persistent goal	/goal	Use for migrations, refactors, or long loops with success criteria.
Trigger review	/review	Use before commit, before PR, or after a fix.
Compact Codex thread	/compact	Use when the transcript becomes too long and noisy.
Drag and drop image into Codex	Hold Shift while dropping	Important in VS Code-like editors where normal drag/drop is intercepted.

Recommended Files at the Root and in Immediate Subfolders

Comparison Table by Purpose, Owner, Update Frequency, and Visibility

The following table groups the recommended core for a modern repository using Cursor and Codex. Some files live directly at the root; others live in root-level tool folders such as `.cursor/` or `.codex/`. The paths shown are either expected by tools or robust conventions for free-form documentation.

File / path	Main role	Typical owner	Update frequency	Recommended visibility
AGENTS.md	Durable shared agent guidance	Tech lead / maintainers	Weekly at first, then as frictions appear	Internal repo, versioned
README.md	Human entry point to the repository	Maintainers	Per release or major change	Public if public repo
ARCHITECTURE.md	System model, boundaries, decisions	Lead dev / architect	At each structural change	Internal or public depending on sensitivity
CONTRIBUTING.md	Contribution and PR rules	Maintainers	Monthly or when process changes	Public if public repo
CODING-STANDARDS.md	Non-trivial coding standards	Lead dev / technical guilds	When real style evolves	Internal, versioned
WORKFLOWS.md	Repeated operating procedures	Team lead / enablement	Frequent at first	Internal, versioned
CODE_REVIEW.md	Review checklist and heuristics	Maintainers / senior reviewers	Monthly	Internal, versioned
TESTING.md	Testing strategy and useful commands	QA / senior developers	When stack evolves	Internal, versioned
PLANS.md or PLAN.template.md	Execution-plan template	Maintainers	Rare; when format matures	Internal, versioned
AGENT-SKILLS.md	Human catalog of in-house skills	AI maintainers / enablement	For each new skill	Internal, versioned
AGENT-MEMORY.md	Stable, non-personal project	Lead dev / maintainers	Short, disciplined,	Internal, versioned

File / path	Main role	Typical owner	Update frequency	Recommended visibility
	memory		reviewed	
.cursor/rules/*.mdc	Cursor-specific rules	Cursor maintainers	When the agent repeats an error	Internal, versioned
.agents/skills/*/SKILL.md	Reusable multi-agent workflows	AI maintainers	When workflow stabilizes	Internal, versioned
.codex/config.toml	Project Codex settings	AI maintainers / dev ops	Rare, strict review	Internal, versioned
.codex/agents/*.toml	Specialized Codex agents	AI maintainers	When specialization matures	Internal, versioned
.cursor/hooks.json	Deterministic Cursor hooks	Tooling maintainers	Rare, strict review	Internal, versioned
.cursorignore	Block agent access to selected files	Security maintainers / lead dev	When repo changes	Internal, versioned
.cursorindexignore	Block indexing and search	Tooling maintainers	When repo changes	Internal, versioned
.editorconfig	Editor consistency, indentation, line endings	Maintainers	Rare	Internal, versioned
CODEOWNERS	Owners by code area	Maintainers / engineering manager	At each reorg or codebase change	Internal/public depending on repo
SECURITY.md	Vulnerability policy	Security maintainers	Per release or policy change	Public if open source
CHANGELOG.md	Human history of changes	Release maintainers	At each release	Public if open source
LICENSE	Repository license	Maintainers	Very rare	Public if open source
CITATION.cff	Academic/ software citation	Maintainers	Rare	Public if relevant

Recommended Minimal Pack

If you do not want to create an over-engineered system, the truly useful minimum is:

AGENTS.md, README.md, ARCHITECTURE.md, CONTRIBUTING.md, WORKFLOWS.md, .cursor/rules/core.mdc, a skills directory such as .agents/skills/, .codex/config.toml,

.cursorignore, .cursorindexingignore, .editorconfig, CODEOWNERS, SECURITY.md, and CHANGELOG.md.

This already covers agent guidance, human onboarding, verification, review, security, and organization.

```
/
├── AGENTS.md
├── README.md
├── ARCHITECTURE.md
├── CONTRIBUTING.md
├── CODING-STANDARDS.md
├── WORKFLOWS.md
├── CODE_REVIEW.md
├── TESTING.md
├── PLANS.md
├── AGENT-SKILLS.md
├── AGENT-MEMORY.md
├── SECURITY.md
├── CHANGELOG.md
├── CODEOWNERS
├── .editorconfig
├── .cursorignore
├── .cursorindexingignore
├── .cursor/
│   ├── rules/
│   │   ├── core.mdc
│   │   └── stack-typescript.mdc
│   └── hooks.json
├── .agents/
│   ├── skills/
│   │   ├── docs-sync/
│   │   │   └── SKILL.md
│   │   └── code-change-verification/
│   │       └── SKILL.md
├── .codex/
│   ├── config.toml
│   └── agents/
│       └── reviewer.toml
```

This tree aligns the recommended separation between shared guidance, Cursor rules, portable skills, and Codex configuration.

Details and Templates by File

Agent Instruction and Configuration Files

AGENTS.md

Purpose. This is the durable source of truth for agents in the repository. Codex reads AGENTS.md before working, supports global and project layers, and Cursor also supports AGENTS.md at the root and in subfolders. Keep it short, concrete, and practical; think of it as a README for agents.

Suggested name. Use AGENTS.md at the root. In mono-repos, you may add additional AGENTS.md files inside package folders when local rules differ.

Format / metadata. Plain Markdown; no frontmatter required. If your internal documentation generator needs metadata, you may add a comment header, but YAML frontmatter is not recommended here because the value of AGENTS.md is simplicity.

Recommended size. 500 to 1500 words in practice, rarely more. If it grows, reference specialized docs. Codex can limit how much of AGENTS.md is loaded using `project_doc_max_bytes`.

Update cadence. Update when the agent makes the same mistake twice, when a standard command changes, or when a workflow becomes stable.

Access. Readable by all developers and agents. Writable by maintainers or through reviewed PRs only. Never store secrets in this file.

Minimal recommended structure

AGENTS.md

Repository overview

- Main stack
- Important directories
- Surfaces to avoid

Basic commands

- Install
- Run
- Lint
- Test
- Typecheck
- Build

Working rules

- Keep public APIs stable

- Add or adjust tests for modified code
- Do not modify generated files
- Do not introduce a new dependency without justification

Definition of done

- Targeted tests pass
- Lint/typecheck pass
- Diff reviewed

Extended recommended structure

AGENTS.md

Repository overview

Important directories

Development commands

Verification commands

Engineering standards

Security rules

Review and PR rules

Reference docs to consult

- ARCHITECTURE.md
- CODING-STANDARDS.md
- WORKFLOWS.md
- CODE_REVIEW.md
- TESTING.md

Definition of done

Local package instructions

Suggested excerpt

Verification commands

- *`pnpm lint`*
- *`pnpm test`*
- *`pnpm typecheck`*
- For a localized change, prefer the smallest relevant subset of tests.

Working rules

- Start with a plan for any multi-file change.
- If a folder convention is unclear, read ARCHITECTURE.md before proposing a structure.
- Refer to WORKFLOWS.md for migration, review, and documentation procedures.

.cursor/rules/.mdc*

Purpose. Encode project-specific Cursor rules: targeted standards, glob-based rules, always-on rules, and manual automation. Cursor documents four application types: Always Apply, Apply Intelligently, Apply to Specific Files, and Apply Manually. OpenAI Codex will not read these files, so they are a complement, not the common source.

Suggested names. Use *.cursor/rules/core.mdc* for Cursor invariants and files such as *.cursor/rules/frontend-react.mdc*, *.cursor/rules/backend-api.mdc*, and *.cursor/rules/tests.mdc* for specific domains.

Format / metadata. Markdown with YAML frontmatter. Important fields include description, globs, and *alwaysApply*. Plain *.md* files in *.cursor/rules/* are not treated as rule files.

Recommended size. Under 500 lines; in practice, avoid more than 100-150 lines per rule. Split rules when they grow.

Update cadence. Add rules only when an error becomes recurrent. This avoids decorative rules that do not improve behavior.

Access. Readable by all, writable under strict review because it changes agent behavior.

Minimal structure

```
---
alwaysApply: true
---
```

- Never modify generated files.
- Run lint and targeted tests after a code change.

Extended structure

```
---
description: React conventions for this repository
globs: src/**/*.tsx
alwaysApply: false
---
```

- Use named exports.
- Keep components small and composable.
- Reuse patterns from *@src/components/Button.tsx*.
- If a component grows, extract subcomponents in the same folder.

Suggested excerpt

```
---
description: Tests and verification for application code
```

*globs: src/**/*.{ts,tsx,js,jsx}*
alwaysApply: false

- Add or update tests for behavior changes.
- Prefer the smallest relevant test subset for fast iteration.
- Report verification commands and results in the final response.

.agents/skills//SKILL.md*

Purpose. Package a reusable workflow with instructions, scripts, references, and optionally assets. Skills keep context clean through progressive loading.

Suggested names. Prefer clear action names: code-change-verification, docs-sync, pr-draft-summary, release-check, migration-planning, incident-triage.

Format / metadata. Markdown with YAML frontmatter. For Codex, name and description are required. Cursor also treats name and description as essential, with optional paths for scope.

Recommended size. Usually 300 to 1200 words, plus optional references/ and scripts/. The description should be concise because it controls implicit activation.

Update cadence. Create a skill when a prompt has been used 3-5 times and the workflow is stable.

Access. Readable by all. Executable scripts should be reviewed strictly. If a skill calls external tools or scripts, treat it as operational code.

Minimal structure

name: docs-sync

description: Update documentation when code changes; use after behavior, command, or configuration changes.

When to use it

- After API, CLI, configuration, or workflow changes

Instructions

- Find the affected docs
- Update the smallest useful surface
- Verify links and command consistency

Extended structure

name: *code-change-verification*

description: *Verify a code change by running lint, targeted tests, typecheck, and summarizing results; use after every non-trivial modification.*

paths:

- "src/**"
- "tests/**"

Expected inputs

Preconditions

Steps

Commands

Success criteria

When to fail loudly

References

Available scripts

Suggested excerpt

Success criteria

- Relevant verification commands pass.
- Known pre-existing failures are distinguished from regressions.
- The final response includes the commands executed and a concise summary of their output.

.codex/config.toml

Purpose. Define project-level Codex behavior: model, sandbox, approvals, MCP, profiles, compaction, memories, and related settings. The same configuration system is used by the CLI and the IDE extension. Project-level `.codex/config.toml` is loaded only when the project is trusted.

Suggested name. `.codex/config.toml` at the project root. Keep personal preferences out of the repo, in `~/.codex/config.toml`.

Format / metadata. TOML, no frontmatter. Important settings include `model`, `approval_policy`, `sandbox_mode`, `[sandbox_workspace_write]`, `[features]`, `project_doc_max_bytes`, `project_doc_fallback_filenames`, `model_auto_compact_token_limit`, `tool_output_token_limit`, and optionally `mcp_servers.*`.

Recommended size. Short and readable, ideally under 100 lines at project level. Complex or personal configuration should live in the user home config or profiles.

Update cadence. Rare. Review strictly. Changing this file changes agent rights and behavior.

Access. Readable by the team. Writable by maintainers only. No secrets should be stored here.

Minimal structure

```
model = "gpt-5.5"  
approval_policy = "on-request"  
sandbox_mode = "workspace-write"
```

[sandbox_workspace_write]

```
network_access = false
```

[features]

```
memories = true
```

Extended structure

```
model = "gpt-5.5"  
approval_policy = "on-request"  
sandbox_mode = "workspace-write"  
model_auto_compact_token_limit = 64000  
tool_output_token_limit = 12000  
project_doc_max_bytes = 32768  
project_doc_fallback_filenames = ["AGENT.md", "TEAM-AGENTS.md"]
```

[sandbox_workspace_write]

```
network_access = false
```

[features]

```
memories = true  
shell_snapshot = true  
goals = true
```

[mcp_servers.docs]

```
enabled = true  
required = false
```

Suggested excerpt

```
# Conservative baseline for team repositories
```

```
approval_policy = "on-request"  
sandbox_mode = "workspace-write"
```

[sandbox_workspace_write]

```
network_access = false
```

[features]

```
goals = true
memories = true
```

.codex/agents/.toml*

Purpose. Define specialized Codex agents. Keep agents narrow and opinionated with a clear job. Each file should define name, description, and developer_instructions.

Suggested names. .codex/agents/reviewer.toml, .codex/agents/pr-explorer.toml, .codex/agents/docs-researcher.toml.

Format / metadata. TOML. Required fields: name, description, developer_instructions. Optional fields may include model, sandbox_mode, mcp_servers, and skills.config.

Recommended size. Very small, usually 20-60 lines. If you need a long document, you are probably writing a skill or workflow doc, not an agent.

Minimal structure

```
name = "reviewer"
description = "Reviews diffs for risks, regressions, and missing tests."
developer_instructions = """
You only perform review.
You do not propose broad refactors.
You focus on correctness, security, tests, and migrations.
"""
```

Extended structure

```
name = "docs-researcher"
description = "Checks compliance with official docs through documentation tools."
model = "gpt-5.4-mini"
sandbox_mode = "read-only"
developer_instructions = """
You validate library, API, and command usage.
You bring precise evidence.
You do not edit code.
"""
```

.cursor/hooks.json

Purpose. Inject deterministic scripts into the Cursor agent loop. Cursor shows examples of stop hooks that rerun the agent until certain criteria are reached. Codex hooks play a similar role and can support logging, validation, secret blocking, or memory generation.

Suggested name. .cursor/hooks.json at the root. Use hooks only for clearly deterministic automation.

Format / metadata. JSON, versioned, with version and hooks.

Recommended size. Small. If a hook grows, move the logic into a versioned script and call it from JSON.

Update cadence. Rare and strictly reviewed. A hook changes the runtime behavior of the agent.

Minimal structure

```
{
  "version": 1,
  "hooks": {
    "stop": [
      { "command": "bun run .cursor/hooks/grind.ts" }
    ]
  }
}
```

Extended structure

```
{
  "version": 1,
  "hooks": {
    "preToolUse": [
      {
        "command": "node .cursor/hooks/pre-write-guard.js",
        "matcher": "Write|StrReplace|Delete",
        "failClosed": true,
        "timeout": 5
      }
    ],
    "stop": [
      { "command": "bun run .cursor/hooks/grind.ts" }
    ]
  }
}
```

Suggested excerpt

```
{
  "version": 1,
  "hooks": {
    "stop": [
      { "command": "node .cursor/hooks/docs-sync.js" }
    ]
  }
}
```

```
}  
}
```

Reserve hooks for cases where success criteria are measurable and controlled.

.cursorignore and .cursorindexingignore

Purpose. Control what Cursor can read and/or index. Use `.cursorignore` for areas that should truly be inaccessible to the agent and `.cursorindexingignore` to reduce noise in semantic search and indexing.

Suggested names. Always use these exact names.

Format / metadata. Ignore-style text patterns. No frontmatter.

Recommended size. Very small and explicit. A few precise patterns are better than a huge opaque list.

Minimal structure

```
# .cursorignore  
.env  
secrets/  
dist/  
coverage/  
  
# .cursorindexingignore  
node_modules/  
dist/  
coverage/
```

Extended structure

```
# .cursorignore  
.env  
.env.*  
secrets/  
private-fixtures/  
artifacts/  
tmp/  
  
# .cursorindexingignore  
node_modules/  
vendor/  
dist/  
coverage/  
storybook-static/  
playwright-report/
```

Human Documentation and Project Memory

README.md

Purpose. The human entry point for the repository. A README is useful for agents too, but it should not replace AGENTS.md: README is for humans, AGENTS is for agents.

Format / metadata. Markdown. No frontmatter is necessary in a raw repository. If your documentation system requires it, use only simple fields such as title, owner, last_review, and audience.

Minimal structure

```
# Project name
## What the project does
## Quickstart
## Useful commands
## Links to detailed docs
```

Extended structure

```
# Project name
## Summary
## Use cases
## Quickstart
## Commands
## Architecture in one page
## Local development
## Contribution
## Security
## License
```

Suggested excerpt

```
## See also
- `AGENTS.md` for repository agent instructions
- `ARCHITECTURE.md` for the system view
- `WORKFLOWS.md` for operational procedures
```

CONTRIBUTING.md

Purpose. Contribution guidelines help contributors and are displayed in several repository surfaces. It is also a good document for agents to read when drafting PRs, branches, commits, or changelogs.

Minimal structure

```
# Contributing
## Prerequisites
## Branching
## Required checks
## Commit / PR format
```

Extended structure

```
# Contributing
## Branch flow
## Local setup
## Quality standards
## Expected tests
## Reviews
## PR templates
## Pre-merge checklist
```

Suggested excerpt

```
## Before opening a PR
- Lint, tests, and typecheck pass
- Changelog updated if needed
- Docs updated if public behavior changes
```

ARCHITECTURE.md

Purpose. This file is not imposed by a tool, but it is the correct place for architectural knowledge that is too detailed for AGENTS.md and too important to remain implicit.

Minimal structure

```
# Architecture
## System boundaries
## Major modules
## Main flows
## Boundaries and dependencies
```

Extended structure

```
# Architecture
## Overview
## Bounded contexts
## Flow diagrams
## Internal / public contracts
## Persistence
## Queues / jobs / events
```

Structural decisions
Sensitive areas

Suggested excerpt

Dependency rule
The domain does not depend on infrastructure.
Adapters depend on the domain, never the reverse.

CODING-STANDARDS.md

Purpose. Collect conventions that are not obvious and not covered well enough by the linter. Avoid copying full style guides into rules; this document is for conventions that are genuinely specific to the repository.

Minimal structure

Coding standards
Naming
File organization
Testing rules
Anti-patterns to avoid

Extended structure

Coding standards
Principles
Naming
Public APIs
Error handling
Logs
Validation
Tests
Canonical examples

Suggested excerpt

Anti-patterns
- No business logic in controllers
- No direct DB access from the front end
- No magic error strings; always use structured errors

WORKFLOWS.md

Purpose. Describe repeated recipes: adding an endpoint, database migration, release, incident, dependency upgrade, reproducible bug fix. This is exactly the kind of knowledge that should later become a skill when stable.

Minimal structure

- # Workflows
 - ## Add a feature
 - ## Fix a bug
 - ## Update documentation

Extended structure

- # Workflows
 - ## Reproducible bug
 - ## Multi-file change
 - ## Migration
 - ## Release
 - ## Documentation
 - ## Local review
 - ## GitHub PR

Suggested excerpt

- ## Reproducible bug
 1. Reproduce
 2. Write or adjust a regression test
 3. Fix with the smallest possible patch
 4. Rerun the reproduction
 5. Run targeted checks

CODE_REVIEW.md

Purpose. Stabilize review behavior across repositories and contributors. This is a strong pattern for Cursor + Codex because review criteria become explicit.

Minimal structure

- # Code review
 - ## Correctness
 - ## Regression risks
 - ## Security
 - ## Tests
 - ## Docs

Extended structure

- # Code review
 - ## Required questions
 - ## Correctness
 - ## Backward compatibility

Security
Performance
Schema / migrations
UX / accessibility
Tests and observability

Suggested excerpt

Always check

- Does the diff change a public contract?
- Is an obvious edge case missing?
- Does the modified code have at least one test or verification proof?

TESTING.md

Purpose. Make the testing strategy executable by humans and agents. Agents become more reliable when they know what “good” looks like.

Minimal structure

Testing
Test pyramid
Main commands
When to add a test
Data sets / fixtures

Extended structure

Testing
Strategy
Unit / integration / e2e
Fast commands
Full commands
Flaky tests
Fixtures
Coverage
Regression policy

PLANS.md

Purpose. Template or registry for execution plans. Useful for long and multi-step work. Agents can write a plan or committed PLAN.md for larger tasks.

Minimal structure

Plan
Goal
Constraints

```
## Steps
## Verification
## Rollback
```

Extended structure

```
# Plan
## Goal
## Out of scope
## Assumptions
## Milestone steps
## Impacted files
## Risks
## Success criteria
## Rollback
## Check results
```

AGENT-SKILLS.md

Purpose. An internal document that catalogs your skills: what they do, when to invoke them, and what inputs they expect. It is not a native tool file, but it becomes useful once you have more than a few skills and want the team to discover them. Reference it from AGENTS.md.

Minimal structure

```
# Agent skills
## docs-sync
## code-change-verification
## release-check
```

Extended structure

```
# Agent skills
## Catalog
### Skill name
- When to use it
- Inputs
- Outputs
- Commands/scripts
- Limits
```

AGENT-MEMORY.md

Purpose. Store stable project facts, not personal memories or stale context. This is a design document distinct from local Codex Memories. It is useful for stable conventions, known pitfalls, canonical directories, and local glossary. Reference it from AGENTS.md; do not assume tools will read it automatically.

Minimal structure

- # Agent memory
- ## Stable facts
- ## Known pitfalls
- ## Do not do

Extended structure

- # Agent memory
- ## Business invariants
- ## Understanding shortcuts
- ## Historical pitfalls
- ## Exceptions to the standard
- ## Local glossary

Suggested excerpt

- ## Known pitfalls
- The *legacy* folder still contains production-used entry points.
- *tenantId* can be null in replay events, even if this is not the nominal case.

Governance and Repository-Hygiene Files

SECURITY.md

SECURITY.md describes supported versions and how to report vulnerabilities. For a public project, it is strongly recommended.

Minimal structure

- # Security policy
- ## Supported versions
- ## How to report a vulnerability
- ## What not to do

Suggested excerpt

Do not report vulnerabilities through a public issue or PR.

CHANGELOG.md

A changelog should be human-readable, grouped by type of change, with versions and dates. It should not be a dump of Git logs. It is especially useful when agents need to draft release notes, migration docs, or PR summaries.

Minimal structure

- # Changelog
- ## [Unreleased]

```
### Added
### Changed
### Fixed
```

CODEOWNERS

CODEOWNERS defines owners for code areas and can automatically request review from them in related PRs. For a team repository, it clarifies who must validate a diff.

Minimal structure

```
* @org/core-team
/src/security/ @org/security-team
/docs/ @org/docs-team
```

.editorconfig

EditorConfig keeps style consistent across editors and IDEs. It prevents agents and humans from applying divergent formatting.

Minimal structure

```
root = true
```

```
[*]
charset = utf-8
end_of_line = lf
insert_final_newline = true
indent_style = space
indent_size = 2
trim_trailing_whitespace = true
```

LICENSE

Include a license at the repository root if the project is open source. It is also useful context for agents generating public docs, templates, or examples.

CITATION.cff

Add this only if your project should be cited academically, in research, or as a software dataset. GitHub supports CITATION.cff at the root, and the CFF format is designed to be readable by both humans and machines.

Operational Checklist and Starter Templates

Adoption Checklist

- ☐ Create a short, concrete AGENTS.md without verbose details. Reference ARCHITECTURE.md, WORKFLOWS.md, CODE_REVIEW.md, TESTING.md, and optionally AGENT-MEMORY.md inside it.
- ☐ Create one very small always-on Cursor rule in .cursor/rules/core.mdc, then add glob-scoped specialized rules if needed.
- ☐ Create only 2 or 3 skills at first: code-change-verification, docs-sync, and either pr-draft-summary or migration-planning.
- ☐ Add .codex/config.toml with a conservative baseline: approval_policy = "on-request", sandbox_mode = "workspace-write", network_access = false, features.goals = true, and features.memories = true.
- ☐ Write WORKFLOWS.md, TESTING.md, and CODE_REVIEW.md before automating. Agents work better when the definition of "good" is explicit.
- ☐ Connect your "code -> test -> review -> PR -> docs" loop before adding too many external tools.
- ☐ Keep conversations short: start a new thread when the task changes; use /compact, /resume, past chats, or a new thread instead of stretching a noisy thread.
- ☐ For any significant change: plan first, milestone next, checks next, review next.

Copy-Ready Starter Pack

AGENTS.md

AGENTS.md

Repository overview

- Application *[briefly describe]*
- Main code in *`src/`*
- Tests in *`tests/`*

Basic commands

- Install: *`pnpm install`*
- Run: *`pnpm dev`*
- Lint: *`pnpm lint`*
- Typecheck: *`pnpm typecheck`*
- Tests: *`pnpm test`*

Working rules

- Start with a plan for any multi-file change.
- Do not modify generated files.
- Preserve public APIs unless explicitly requested.
- Add or adjust tests if behavior changes.

- Use the smallest relevant set of checks for fast iteration.

Docs to consult

- ``ARCHITECTURE.md``
- ``CODING-STANDARDS.md``
- ``WORKFLOWS.md``
- ``TESTING.md``
- ``CODE_REVIEW.md``
- ``AGENT-MEMORY.md``

Definition of done

- Targeted tests pass
- Lint/typecheck pass
- Diff reviewed
- Verification instructions communicated

`.cursor/rules/core.mdc`

alwaysApply: true

- Respect ``AGENTS.md`` and the documents referenced from it.
- Never modify generated files.
- For a behavior change, add or update tests.
- Report the verification commands executed.

`.agents/skills/code-change-verification/SKILL.md`

name: code-change-verification

description: Verify a code change by running the relevant checks; use after any non-trivial modification.

Preconditions

- The repository is installed and verification scripts exist.

Steps

1. Identify the impacted area.
2. Choose the smallest relevant set of checks.
3. Run lint, tests, or typecheck depending on the change.
4. Summarize the results.
5. If there is a failure, propose the minimal correction or report the blocker.

Success criteria

- Targeted checks pass
- Results are reported clearly

.codex/config.toml

```
model = "gpt-5.5"
approval_policy = "on-request"
sandbox_mode = "workspace-write"
model_auto_compact_token_limit = 64000
tool_output_token_limit = 12000
project_doc_max_bytes = 32768
```

[sandbox_workspace_write]

```
network_access = false
```

[features]

```
goals = true
memories = true
shell_snapshot = true
```

WORKFLOWS.md

Workflows

Bug fix

1. Reproduce the bug
2. Write or adjust a regression test
3. Fix with the smallest possible patch
4. Rerun the reproduction
5. Run targeted checks
6. Launch a local review

Multi-file change

1. Write a plan
2. Split into milestones
3. Implement one milestone at a time
4. Verify between each milestone
5. Update docs continuously

CODE_REVIEW.md

Code review

Correctness

- Does the code do exactly what was requested?

Risks

- Forgotten edge cases?
- Possible regression?
- Public contract impacted?

Security

- Permissions, validation, sensitive data?

Tests

- Is the change covered?
- Were the relevant checks executed?

Docs

- README / docs / changelog to update?

AGENT-MEMORY.md

Agent memory

Stable facts

- *`legacy`* still contains critical paths.
- *`tenantId`* can be null in some replay events.

Known pitfalls

- DB migrations must be backward compatible.
- The front end still consumes the old *`statusLabel`* field in some screens.

Limits and Points of Caution

Some Cursor features evolve quickly, especially run modes, hooks, and shortcuts. The recommendations above are based on currently inspectable official documentation and recent OpenAI and Cursor materials, but surface details - exact mode names, availability by version, and advanced plugin options - may evolve faster than the working principles described here.

The principles are stable: short and versioned guidance, precise context, reusable workflows, strict permissions, systematic tests, and systematic review.